Computational Logic in the Era of AI

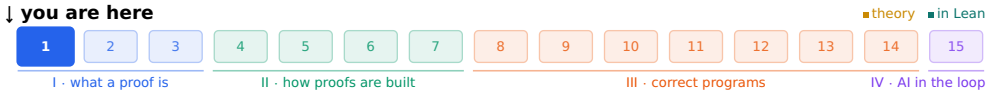
Untyped Lambda Calculus

Sergey Mechtaev

mechtaev@pku.edu.cn

Term 1 • 26–27

This lecture



1 **Rules**

inference rules, derivation trees, the closure clause

`inductive`, `Term.rec`

► Rules define a system, and a derivation is the only evidence that a term belongs to it.

2 **Terms**

Λ; binding, α-equivalence, de Bruijn

`Term`, `FV`, `DB`

► Proofs and programs are stored as trees like this; bound names never matter.

3 **Substitution**

$M[x := N]$, and how capture is avoided

`subst`, `rename`

► Instantiating a ∀ is substitution; capture would make a checker unsound.

4 **Computation**

β-reduction, normal forms, confluence

`step`, `nf`

► Equality by computation: both sides reduce to the same normal form, uniquely.

5 **Recursion**

fixed points, Y, Church numerals

`Y`, `church`

► Unrestricted recursion makes termination undecidable; types are the fix.

History: one question, two answers

1928. Hilbert's *Entscheidungsproblem*: is there a procedure that decides whether any given statement of logic is provable? To answer no, one must first say what a procedure is.

David Hilbert

1862–1943

Poses the problem, expecting yes.

Alonzo Church

1903–1995

The λ-calculus, 1932. In 1936 it defines "procedure", and the answer is no.

Alan Turing

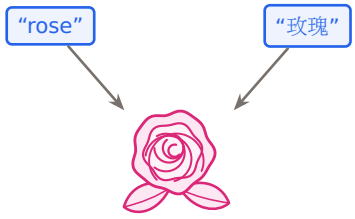
1912–1954

Turing machines, 1936: the same answer, and the two notions of procedure coincide.

The calculus began as a foundation for logic, was found inconsistent (Kleene and Rosser, 1935), and survived as a theory of computation — the one this lecture builds. Photos: Wikimedia.

Language and meta-language

> “What’s in a name?
> That which we call a rose
> By any other name would smell as sweet.”
>
> — Juliet, *Romeo and Juliet*, II.ii

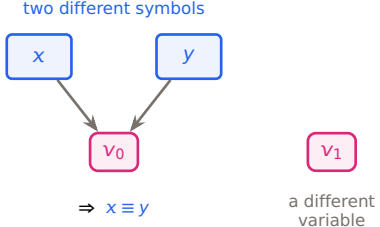


two names, one flower

object language — what we study: the λ-terms.

meta-language — what we use to talk about them: x, M, $\equiv$, and the rules themselves.

So two meta-variables are two symbols, and may still stand for one and the same object:



Inductive definition of λ-terms

Variables are inexhaustible: $\mathrm{Var} = \{v_0, v_1, v_2, \ldots\}$. The rest of the alphabet is λ, $.$, $($ and $)$.

$$\frac{x \in \mathrm{Var}}{x \in \Lambda}\ (\text{var}) \qquad \frac{M \in \Lambda \quad N \in \Lambda}{(M\,N) \in \Lambda}\ (\text{app}) \qquad \frac{x \in \mathrm{Var} \quad M \in \Lambda}{(\lambda x.\, M) \in \Lambda}\ (\text{abs})$$

Λ is the *smallest* set closed under the three rules — and nothing else is a λ-term. Equivalently, as a grammar:

$$\Lambda \;=\; \mathrm{Var} \mid (\Lambda\Lambda) \mid (\lambda \mathrm{Var}.\, \Lambda)$$

pink the term itself, blue metavariables: M, N over Λ, x, y over Var. N&G §1.3, Def. 1.3.2, §2.4

Membership is a derivation

$$\dfrac{\dfrac{\dfrac{v_0 \in \mathsf{Var}}{v_0 \in \Lambda}\,(\text{var}) \qquad \dfrac{v_1 \in \mathsf{Var}}{v_1 \in \Lambda}\,(\text{var})}{(v_0\ v_1) \in \Lambda}\,(\text{app})}{(\lambda v_0.\,(v_0\ v_1)) \in \Lambda}\,(\text{abs})$$

$M \in \Lambda$ *means* that some finite tree of rule instances has M at its root.

Lean’s def, #eval, #check

`def` names a value

`#eval` runs it

`#check` asks its type — of any expression

```
def double (n : Nat) : Nat := 2 * n

#eval double 21          -- 42

#check (double)          -- double : Nat → Nat

#check double 21         -- double 21 : Nat
```

Two arguments are two arrows: a function returning a function.

```
def tag (k : Nat) (s : String) : String :=
  s ++ toString k
#check (tag)             -- tag : Nat → String → String
#eval tag 3 "v"          -- "v3"
```

Nat is an inductive type

Two ways to build a natural number, and nothing else:

$$\frac{}{0 : \mathtt{Nat}}\ \text{(zero)} \qquad \frac{n : \mathtt{Nat}}{n + 1 : \mathtt{Nat}}\ \text{(succ)}$$

```
#print Nat
-- inductive Nat : Type
-- constructors:
--   Nat.zero : Nat
--   Nat.succ : Nat → Nat
```

Two shapes, so a definition by cases has two clauses, and a recursive call may only go to the smaller one.

Pattern matching, and what Lean demands

accepted
both shapes covered;
recursion on a smaller n

```
def fact : Nat → Nat
  | 0     => 1
  | n + 1 => (n + 1) * fact n
#eval fact 5            -- 120
```

rejected
a shape is missing:
`Nat.succ Nat.zero`

```
def half : Nat → Nat
  | 0     => 0
  | n + 2 => half n + 1
```

rejected
nothing gets smaller

```
def loop (n : Nat) : Nat := loop (n + 1)
```

So every `def` is a total function: defined everywhere, and it stops.

Λ is an inductive type

```
inductive Term where
  | var : String → Term
  | app : Term → Term → Term
  | lam : String → Term → Term
```

The three rules, transcribed: three constructors, and no fourth way in.

Examples of λ-terms

on paper

x

$(x\,y)$

$(\lambda x.\,(x\,y))$

$((\lambda x.\,x)(\lambda x.\,x))$

in Lean

```
[ var "x",
  app (var "x") (var "y"),
  lam "x" (app (var "x") (var "y")),
  app (lam "x" (var "x")) (lam "x" (var "x")) ]
```

Syntactic identity ($\equiv$)

When are two λ-terms "the same"? The strictest answer:

Definition (syntactic identity, $\equiv$)

$M \equiv N$ means M and N are the *very same term* — identical symbol for symbol (equivalently, the same tree).

- $(v_0\, v_2) \equiv (v_0\, v_2)$ the same term;
- $(v_0\, v_2) \not\equiv (v_0\, v_1)$ differ: v_2 vs. v_1.

$\equiv$ lives in the meta-language

It is a statement we make *about* terms; it never appears *inside* a term; "$x \not\equiv y$" simply says the two variables are different.

Syntactic identity is decidable

≡ compares names, so a program can check it: `deriving DecidableEq` supplies the algorithm, == runs it, and `decide` turns its verdict into a proof.

```
#eval I == I                                        -- true
#eval lam "x" (var "x") == lam "z" (var "z")        -- false: the names differ

example : lam "x" (var "x") ≠ lam "z" (var "z") := by decide
```

Two terms that differ only in a bound name are distinct under ≡ — the defect α-conversion repairs.

Subterms

Definition (Sub, a multiset)

1. $\mathrm{Sub}(x) = \{x\}$;
2. $\mathrm{Sub}((M\,N)) = \mathrm{Sub}(M) \cup \mathrm{Sub}(N) \cup \{(M\,N)\}$;
3. $\mathrm{Sub}((\lambda x.\, M)) = \mathrm{Sub}(M) \cup \{(\lambda x.\, M)\}$.

L is a *subterm* of M if $L \in \mathrm{Sub}(M)$; a *proper* subterm if moreover $L \not\equiv M$.

A term can occur several times — as different *occurrences*. For example,

$$\mathrm{Sub}\big((\lambda x.\,(x\,x))\big) = \big\{\ (\lambda x.\,(x\,x)),\quad (x\,x),\quad x,\quad x\ \big\}:$$

four subterms — the whole term, its body $(x\,x)$, and *two* occurrences of x. The binding x just after λ does *not* count.

Saving parentheses

Fully parenthesised terms are unreadable. Conventions:

- Outermost parentheses may be dropped: $M\,N$ for $(M\,N)$.
- Application is **left-associative**: $M\,N\,L \equiv ((M\,N)\,L)$.
- Application binds tighter than abstraction: $\lambda x.\, M\,N \equiv \lambda x.\, (M\,N)$.
- Successive λ's combine, **right-associatively**: $\lambda xy.\, M \equiv \lambda x.\, (\lambda y.\, M)$.

Treacherous

$\lambda v_0.\, v_0\,(v_1\,v_0)$ is *not* $(\lambda v_0.\, v_0)(v_1\,v_0)$, but $\lambda v_0.\, \big(v_0\,(v_1\,v_0)\big)$.

Three roles of a variable occurrence

- **Binding**: the occurrence immediately after a λ.
- **Bound**: an occurrence captured by some enclosing λ.
- **Free**: everything else.

Abstraction of x over M *binds* all free occurrences of x in M.

Free variables FV

1. $\mathrm{FV}(x) = \{x\}$;
2. $\mathrm{FV}(M\,N) = \mathrm{FV}(M) \cup \mathrm{FV}(N)$;
3. $\mathrm{FV}(\lambda x.\, M) = \mathrm{FV}(M) \setminus \{x\}$.

In $x(\lambda x.\, x\, y)$ both x and y are free *somewhere*: the first x is free, but the x before y is bound. So FV records the variables free *at some occurrence*.

Closed terms and combinators

Definition

A term M is *closed* if $\mathrm{FV}(M) = \varnothing$. A closed term is also called a *combinator*. The set of closed terms is Λ^0.

- $\lambda xyz.\, x\, x\, y$ and $\lambda xy.\, x\, x\, y$ are closed;
- $\lambda x.\, x\, x\, y$ is *not* (y is free).

FV, by recursion over the rules

One clause per constructor, as in (Nederpelt and Geuvers 2014a, §1.4).

```
def FV : Term → List String
  | var x   => [x]                          -- a variable is free in itself
  | app P Q => nameUnion (FV P) (FV Q)      -- free on either side
  | lam x P => (FV P).erase x               -- x is bound here

#eval Term.FV (lam "x" (app (var "x") (var "y")))   -- ["y"]
```

Renaming bound variables

The name of a binding variable is not essential: $\lambda x.\, x^2$ and $\lambda u.\, u^2$ describe the *same* function.

Definition (renaming, $=_\alpha$)

Let $M^{x \to y}$ replace every *free* x in M by y. Then

$$\lambda x.\, M =_\alpha \lambda y.\, M^{x \to y}$$

provided (i) $y \notin \mathrm{FV}(M)$ and (ii) y is not a binding variable in M.

Why (i)? $\lambda x.\, y \not=_\alpha \lambda y.\, y$: a free y would be captured.

Why (ii)? $\lambda x.\, \lambda y.\, x \not=_\alpha \lambda y.\, \lambda y.\, y$: an inner λy would capture the new y.

α-equivalence

Extend renaming to an equivalence relation by closing under:

- **Compatibility**: if $M =_\alpha N$ then $ML =_\alpha NL$, $LM =_\alpha LN$, and $\lambda z.\, M =_\alpha \lambda z.\, N$ (so we may rename inside subterms);
- **Reflexivity**, **Symmetry**, **Transitivity**.

Examples

$(\lambda x.\, x\,(\lambda z.\, x\,y))\,z =_\alpha (\lambda u.\, u\,(\lambda z.\, u\,y))\,z$	✓
$(\lambda x.\, x\,(\lambda z.\, x\,y))\,z \not=_\alpha (\lambda y.\, y\,(\lambda z.\, y\,y))\,z$	(y captured, cond. i)
$(\lambda x.\, x\,(\lambda z.\, x\,y))\,z \not=_\alpha (\lambda z.\, z\,(\lambda z.\, z\,y))\,z$	(z is binding, cond. ii)

If $M =_\alpha N$, then M and N are *α-variants*.

Capture-avoiding substitution

$M[x := N]$ = "M with N substituted for the free x".

Definition

(1a) $x[x := N] \equiv N$;

(1b) $y[x := N] \equiv y$ if $x \not\equiv y$;

(2) $(P\,Q)[x := N] \equiv (P[x := N])(Q[x := N])$;

(3) $(\lambda y.\, P)[x := N] \equiv \lambda z.\, (P^{y \to z})[x := N]$, choosing $\lambda z.\, P^{y \to z} =_\alpha \lambda y.\, P$ with $z \notin \mathrm{FV}(N)$.

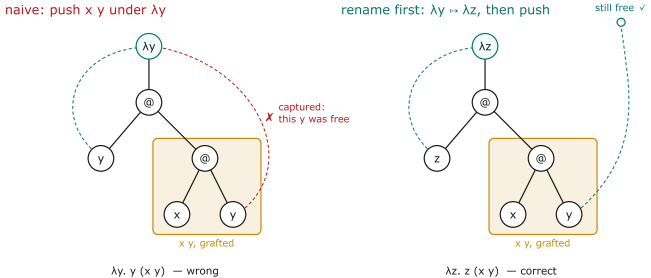
The point of clause (3)

Rename the bound y first so that a free y inside N is *not captured* when pushed under λy.

$M[x := N]$ is meta-notation: apply the rules until all brackets disappear, yielding a genuine λ-term.

Substitution: capture in action

$(\lambda y.\, y\, x)[x := x\, y]$: the argument $x\, y$ carries a *free* y, and it is about to be pushed under a λy.



Clause (3): the bound name is renamed away from FV(N) before N is pushed under the binder.

Every bound occurrence points back at its binder; capture is a *new* arrow where none should be. Renaming the binder first leaves the argument's arrows alone: $\lambda z.\, (z\, x)[x := x\, y] \equiv \lambda z.\, z\, (x\, y)$.

Substitution, and the clause that avoids capture

(Nederpelt and Geuvers 2014a, §1.6): the first two clauses are bookkeeping; the third carries the definition.

```
def subst (x : String) (N : Term) : Term → Term
  | var y   => if y = x then N else var y
  | app P Q => app (subst x N P) (subst x N Q)
  | lam y P =>
      if y = x then lam y P                -- x is bound here: stop
      else if (FV N).contains y then
        let z := fresh (nameUnion (FV N) (FV P)) y
        lam z (subst x N (rename y z P))   -- rename y apart, descend
      else lam y (subst x N P)
termination_by M => M.size
```

Clause 3 recurses on `rename y z P`, not on a subterm, so the recursion is not structural: we say what shrinks — the size — and Lean checks it. In full in L5.

Capture, prevented

Two slides ago the fix was to rename the binder before descending; the last slide wrote that fix into clause 3. Run `subst` on that example, $(\lambda y.\, y\, x)[x := x\, y]$:

```
#eval subst "x" (app (var "x") (var "y"))
                (lam "y" (app (var "y") (var "x")))
-- lam "y'" (app (var "y'") (app (var "x") (var "y")))
```

on paper	$\lambda z.\, z\,(x\, y)$
Lean	$\lambda y'.\, y'\,(x\, y)$

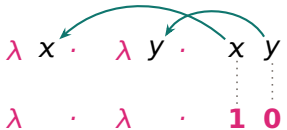
Clause 3 fired: the binder was renamed, and the argument's y is still free. Only the fresh name differs, and which fresh name is chosen is immaterial — α-equivalence is exactly what makes "immaterial" precise.

Nameless terms: the de Bruijn idea

α-conversion is a way to *cope* with variable names. The **de Bruijn representation** (1972) instead *eliminates* them: every variable becomes a number.

The rule

Replace each *bound* occurrence by the count of λ's you cross, going outward, to reach its own binder. The *nearest* enclosing λ is **0**, the next **1**, and so on.



The right-hand x crosses one inner λ (the λy) to reach its binder $\Rightarrow$ **1**; the y reaches its binder immediately $\Rightarrow$ **0**.

The object/meta colour has done its job: from here on a name is only a representative of its α-class, so we reason in plain black, keeping the pink λ-skeleton as a structural cue.

De Bruijn indices: examples

	named	nameless
I	$\lambda x.\, x$	$\lambda.\, \mathbf{0}$
K	$\lambda x.\, \lambda y.\, x$	$\lambda.\, \lambda.\, \mathbf{1}$
S	$\lambda x.\, \lambda y.\, \lambda z.\, (x\, z)(y\, z)$	$\lambda.\, \lambda.\, \lambda.\, (\mathbf{2}\, \mathbf{0})(\mathbf{1}\, \mathbf{0})$
ω	$\lambda x.\, x\, x$	$\lambda.\, \mathbf{0}\, \mathbf{0}$

α-equivalence collapses to identity

$\lambda x.\, x$, $\lambda y.\, y$ and $\lambda z.\, z$ all map to the *same* nameless term $\lambda.\, \mathbf{0}$. So on de Bruijn terms, $\equiv$ *is* α-equivalence — renaming and capture-checking both disappear.

Free variables: either read them off a fixed context — under $\Gamma = w$, $\lambda x.\, w\, x \mapsto \lambda.\, \mathbf{1}\, \mathbf{0}$, where $\mathbf{1}$ points *past* the outermost λ into Γ — or, with Γ empty, let them keep their *name*. `toDB` on the next slide does both: the context is its argument.

Erasing names decides $=_\alpha$

(Nederpelt and Geuvers 2014a, §1.12). A bound variable becomes the number of binders between it and its λ; a free one is looked up in the context, or keeps its name.

```
#eval toDB [] (lam "x" (var "x"))          -- lam (idx 0)
#eval toDB [] (lam "z" (var "z"))          -- lam (idx 0)   -- same
#eval toDB ["w"] (lam "x" (app (var "w") (var "x")))
-- lam (app (idx 1) (idx 0))     -- w: read off the context
#eval toDB []    (lam "x" (app (var "w") (var "x")))
-- lam (app (free "w") (idx 0))  -- w: keeps its name

#eval alphaEq (lam "x" (var "x")) (lam "z" (var "z"))  -- true
#eval alphaEq (var "a") (var "b")                      -- false
```

$=_\alpha$ becomes a computation, not a search for a renaming. Every clause of `toDB`, `DB.subst` and `DB.step` is structural, so `decide` can run them — as it could not the size-recursive named `subst`. Bruijn (1972).

One-step β-reduction

Definition ($\to_\beta$)

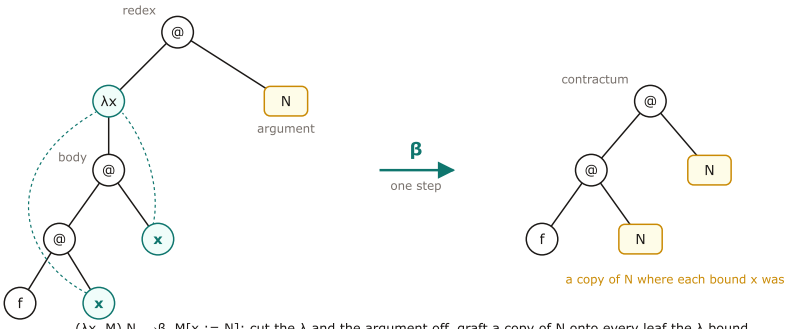
1. (Basis) $(\lambda x.\, M)\, N \to_\beta M[x := N]$;
2. (Compatibility) if $M \to_\beta N$ then $ML \to_\beta NL$, $LM \to_\beta LN$, and $\lambda x.\, M \to_\beta \lambda x.\, N$.

- A subterm $(\lambda x.\, M)\, N$ is a **redex** (*red*ucible *ex*pression);
- $M[x := N]$ is its **contractum**;
- M in $\lambda x.\, M$ is the **body**.

One-step: exactly one redex is replaced by its contractum.

A β-step is tree surgery

The basis clause, drawn on the trees. $(\lambda x.\, f\, x\, x)\, N \to_\beta f\, N\, N$:



(λx. M) N →β M[x := N]: cut the λ and the argument off, graft a copy of N onto every leaf the λ bound.

The λ and the argument are cut off; the body survives, with a copy of N grafted onto every leaf the λ bound. Substitution tracks *which* leaves; compatibility lets the surgery happen anywhere inside a larger tree.

Reduction can branch

The term $(\lambda x.\,(\lambda y.\,y\,x)z)\,v$ has *two* redexes:

$$(\lambda x.\,(\lambda y.\,y\,x)z)\,v \;\to_\beta\; (\lambda y.\,y\,v)\,z \qquad \text{(outer redex)}$$
$$(\lambda x.\,(\lambda y.\,y\,x)z)\,v \;\to_\beta\; (\lambda x.\,z\,x)\,v \qquad \text{(inner redex)}$$

The two results differ — but both reduce on to a **common reduct** $z\,v$:

$$(\lambda y.\,y\,v)\,z \to_\beta z\,v \qquad \text{and} \qquad (\lambda x.\,z\,x)\,v \to_\beta z\,v.$$

Self-application: a term that reduces to itself

$\Omega := (\lambda x.\,x\,x)(\lambda x.\,x\,x) \to_\beta \Omega \to_\beta \Omega \to_\beta \cdots$

One step, made deterministic

(Nederpelt and Geuvers 2014a, §1.8) defines $\rightarrow_\beta$ as a *relation*: a term may have several redexes. Running it requires choosing one.

```
-- Def. 1.8.1 gives →β as a *relation*: a term may have several
-- redexes.  To run it we must choose one.  This is
-- leftmost-outermost.
def step : Term → Option Term
  | app (lam x P) Q => some (subst x Q P)    -- the basis: contract
  | app P Q         => match step P with       -- else look left,
      | some P' => some (app P' Q)
      | none    => (step Q).map (app P)        -- then right,
  | lam x P         => (step P).map (lam x)    -- then under the λ.
  | var _           => none                  -- a variable is normal
```

Leftmost-outermost. Confluence says the choice cannot change the answer — only whether we find it.

Multi-step reduction and conversion

β-reduction $\twoheadrightarrow_\beta$ (zero-or-more steps)

$M \twoheadrightarrow_\beta N$ if there is a chain $M \equiv M_0 \to_\beta M_1 \to_\beta \cdots \to_\beta M_n \equiv N$ $(n \geq 0)$.
It is reflexive and transitive, and extends $\to_\beta$.

β-conversion $=_\beta$ (the equivalence)

$M =_\beta N$ if M and N are linked by a chain of steps taken in *either* direction. It is reflexive, symmetric, and transitive.

E.g. all four terms below are pairwise $=_\beta$:

$$\{\, (\lambda x.\,(\lambda y.\,y\,x)z)\,v,\ (\lambda y.\,y\,v)\,z,\ (\lambda x.\,z\,x)\,v,\ z\,v \,\}.$$

Normal forms — the “outcome” of a computation

Definition

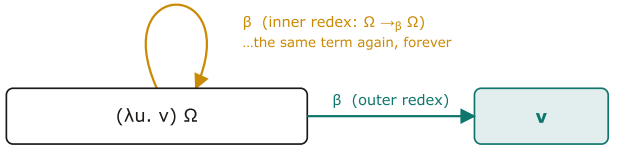
- M is in *β-normal form* if it contains *no redex*.
- M *has* a normal form (M is *normalising*) if $M =_\beta N$ for some N in normal form.

Weak vs. strong normalisation

- **Weakly** normalising: *some* reduction path reaches a normal form.
- **Strongly** normalising: *no* infinite reduction path exists (every path terminates). Strong ⇒ weak.

Weak versus strong, on one term

$(\lambda u.\, v)\,\Omega$ has two redexes, and they lead to two different fates:

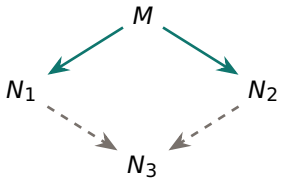


The reduction graph of a term is what normalisation talks about: *weakly* normalising means some path out of the node ends; *strongly* normalising means every path does. Ω alone is a node whose only edge is the loop — no path ends, so no normal form exists, which is why `nf` on the Lean rail takes a budget.

Church–Rosser / Confluence

Theorem 1.9.8 (Church–Rosser, Confluence)

If $M \twoheadrightarrow_\beta N_1$ and $M \twoheadrightarrow_\beta N_2$, then there exists N_3 with $N_1 \twoheadrightarrow_\beta N_3$ and $N_2 \twoheadrightarrow_\beta N_3$.



Consequences

The outcome of a computation, if it exists, is unique — independent of reduction order.

Why the budget is not an artefact

`nf` takes a step count. By (Nederpelt and Geuvers 2014a, §1.9) some terms have no normal form, so *no* total function returns one.

```
-- ‘nf‘ takes a step budget.  This is not a weakness of Lean: by
-- §1.9 some terms have *no* normal form, so no total function
-- returns one.
#eval nf 20 (app I (var "q"))                    -- var "q"
#eval nf 20 (app (app K (var "a")) (var "b"))    -- var "a"

-- Ω reduces to itself forever; after 20 steps we are where we
-- began.
#eval nf 20 Ω == Ω                               -- true
#eval nf 1000 Ω == Ω                             -- true
```

Lean insists every `def` is total; the fuel is where that insistence becomes visible.

Every term has a fixed point

Theorem

For every $L \in \Lambda$ there is $M \in \Lambda$ with $L\,M =_\beta M$.

Proof. Put $M := (\lambda x.\, L(x\,x))(\lambda x.\, L(x\,x))$. Then

$$M \equiv (\lambda x.\, L(x\,x))(\lambda x.\, L(x\,x)) \to_\beta L\big((\lambda x.\, L(x\,x))(\lambda x.\, L(x\,x))\big) \equiv L\,M. \qquad \square$$

The fixed-point combinator Y

A single closed term builds a fixed point for *any* input:

$$Y \;\equiv\; \lambda y.\, (\lambda x.\, y(x\,x))(\lambda x.\, y(x\,x)), \qquad L(Y\,L) =_\beta Y\,L.$$

Contrast ordinary calculus: the successor $n \mapsto n+1$ has *no* fixed point. Untyped λ-calculus does.

Fixed points give recursion

A recursive equation $M =_\beta \ldots M \ldots$ always has a solution: let $L \equiv \lambda z.\ \ldots z \ldots$ (replace M by a fresh z); then any fixed point of L solves the equation — and $Y\,L$ is one.

Example: the factorial

Encoding numbers, `if-then-else`, `iszero`, `mult`, `pred` as λ-terms, the equation

$$\mathtt{fac}\,x =_\beta \mathtt{if}\ (\mathtt{iszero}\ x)\ \mathtt{then}\ \mathtt{1}\ \mathtt{else}\ \mathtt{mult}\ x\ (\mathtt{fac}\,(\mathtt{pred}\ x))$$

is solved by a fixed point of the corresponding L — i.e. by $Y\,L$.

This is how unbounded recursion arises with no built-in recursion construct.

Fixed points, computed

(Nederpelt and Geuvers 2014a, Thm. 1.10.1): every term has a fixed point, and *Y* constructs it.

```
-- Thm. 1.10.1: every term has a fixed point.  Curry's Y builds it.
--   Y f  ↠β  f (Y f)
#eval nf 3 (app Y (var "f"))
-- app (var "f") (app (var "f") (app (lam "x" ..) (lam "x" ..)))
-- Each further step peels off one more 'f': the term never settles,
-- which is what "Y f is a fixed point of f" means operationally.

-- This is why the untyped calculus is Turing-complete, and why
-- every λ-term being typable would be *too much* to ask: 'Y' cannot
-- be typed.
#eval Term.isClosed Y                          -- true
```

Numbers, from nothing but functions

A number is how many times you do something. The numeral $\ulcorner n \urcorner$ takes a function f and a start x, and applies f to x n times:

$$\begin{aligned}
\ulcorner 0 \urcorner &\equiv \lambda f.\, \lambda x.\, x \\
\ulcorner 1 \urcorner &\equiv \lambda f.\, \lambda x.\, f\, x \\
\ulcorner 2 \urcorner &\equiv \lambda f.\, \lambda x.\, f\, (f\, x) \\
\ulcorner 3 \urcorner &\equiv \lambda f.\, \lambda x.\, f\, (f\, (f\, x))
\end{aligned} \qquad \ulcorner n \urcorner\, f\, x =_\beta f^n\, x$$

$\ulcorner 3 \urcorner\, f\, x$: three passes through f

Successor adds one pass: $\mathsf{succ} \equiv \lambda n.\, \lambda f.\, \lambda x.\, f\, (n\, f\, x)$, so $\mathsf{succ}\, \ulcorner 1 \urcorner \to_\beta \lambda f.\, \lambda x.\, f\, (\ulcorner 1 \urcorner\, f\, x) \twoheadrightarrow_\beta \lambda f.\, \lambda x.\, f\, (f\, x) \equiv \ulcorner 2 \urcorner$.

Addition composes iterators

To add, do n's passes and then m's on top of them:

$$\mathrm{add} \equiv \lambda m.\,\lambda n.\,\lambda f.\,\lambda x.\; m\,f\,(n\,f\,x) \qquad \text{so} \qquad \mathrm{add}\ulcorner 2\urcorner\ulcorner 3\urcorner\,f\,x \twoheadrightarrow_\beta f\,(f\,(f\,(f\,(f\,x))))$$

```
def add : Term :=
  lam "m" (lam "n" (lam "f" (lam "x"
    (app (app (var "m") (var "f"))
         (app (app (var "n") (var "f")) (var "x"))))))

#eval alphaEq (nf 100 (app (app add (church 2)) (church 3)))
              (church 5)                          -- true
```

No primitive numbers, yet $2 + 3 = 5$ falls out of β-reduction; with Y, so does the factorial. (Nederpelt and Geuvers 2014a, §1.10), Exercises 1.10–1.14.

What untyped λ-calculus gives us, and what it costs

Kept

- a clean formalism for building and evaluating functions;
- rigorous, capture-avoiding substitution;
- conversion = “equal by calculation”; normal forms = outcomes;
- confluence (Nederpelt and Geuvers 2014a, Thm. 1.9.8) ⇒ outcomes are unique;
- recursion via fixed points (Nederpelt and Geuvers 2014a, Thm. 1.10.1);
- **Turing-complete**: λ-definable = Turing-computable (Nederpelt and Geuvers 2014a, §1.12).

Two defects, both visible in Ω and in Y

1. *Self-application is legal.* $\lambda x.\, x\,x$ applies a term to itself; nothing in the grammar objects, and Y is built from it.
2. *Computation need not stop.* $\Omega \to_\beta \Omega \to_\beta \cdots$: no normal form, so no total function can compute one.

What chapter 2 changes

Types outlaw $x\,x$ *and* make every legal term terminate — Strong Normalisation (Nederpelt and Geuvers 2014a, Thm. 2.11.6). The price: Y becomes untypable, and Turing-completeness is lost.

That trade — *lose arbitrary recursion, gain a decidable check* — is the first of three facts behind “why is a Lean proof trusted?” L2 adds the second, L3 assembles all three.

References I

Bruijn, N. G. de (1972). “Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church–Rosser theorem”. In: *Indagationes Mathematicae* 34, pp. 381–392.

Christiansen, D. T. (2026). *Functional Programming in Lean*. Copyright Microsoft Corporation 2023 and Lean FRO, LLC 2023–2026; tested against Lean 4.33.0. URL: https://lean-lang.org/functional_programming_in_lean/ (visited on 08/20/2026).

Nederpelt, R. and H. Geuvers (2014a). *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press.

— (2014b). “Untyped lambda calculus”. In: *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press. Chap. 1, pp. 1–32.

References II

The three Lean slides follow *Functional Programming in Lean*, ch. 1 (`def`, `#eval`, `#check`) and ch. 2 (pattern matching, structural recursion on `Nat` and `List`). Chapter 1 of the book throughout: §1.3 terms (Def. 1.3.2, Notation 1.3.4), §1.4 free variables (Def. 1.4.1) and combinators (Def. 1.4.3), §1.5 α-conversion (Def. 1.5.2), §1.6 substitution (Def. 1.6.1), §1.8 β-reduction (Def. 1.8.1), §1.9 normal forms, Church–Rosser (Thm. 1.9.8) and uniqueness of normal forms (Lem. 1.9.10(2)), §1.10 the Fixed Point Theorem (Thm. 1.10.1). Church numerals and de Bruijn indices are not developed in the chapter body: they appear in Exercises 1.10–1.14 and in §1.12 respectively.